

MODULUS

Retail Operations Platform

CI/CD Pipeline Specification

Version 1.0

1. Overview

Each of the four Modulus applications has an independent GitHub Actions pipeline. A change to one application triggers only that application's pipeline. No pipeline is shared. This independence is what makes the micro-frontend architecture operationally meaningful: teams ship without coordination.

All pipelines follow the same stage sequence: checkout, install, lint, typecheck, unit test, build, and deploy. A failure at any stage terminates the pipeline immediately. Subsequent stages do not run. The deployment stage only executes if all preceding stages pass.

The shell pipeline has one additional stage appended after deploy: trigger the Playwright E2E suite against the live composed application. This stage is exclusive to the shell pipeline because the shell is the composition point for all remotes.

2. Pipeline Triggers

Workflow File	Application	Trigger Path	Trigger Branch
shell.yml	modulus-shell	apps/shell/**	main
inventory.yml	modulus-inventory	apps/inventory/**	main
orders.yml	modulus-orders	apps/orders/**	main
analytics.yml	modulus-analytics	apps/analytics/**	main

Pipelines also trigger on pull requests targeting main. Pull request runs perform all stages except the deploy stage. This validates the change before it reaches the production branch.

Changes to packages/* do not trigger any application pipeline directly. When a shared package changes, the developer must push a corresponding change to each affected application to trigger its pipeline. This is intentional: it makes shared package adoption explicit and traceable.

3. Stage Definitions

3.1 Checkout

```
- uses: actions/checkout@v4
```

Checks out the repository at the commit that triggered the pipeline. No additional configuration required.

3.2 Install

```
- uses: actions/setup-node@v4
  with:
    node-version: "20"
    cache: "npm"
- run: npm ci
```

Uses npm ci rather than npm install to ensure reproducible installs from package-lock.json. The node_modules cache is keyed on package-lock.json to avoid redundant installs on unchanged dependencies.

3.3 Lint

```
- run: npx turbo run lint --filter=<app-name>
```

Runs ESLint against the target application only. The --filter flag restricts Turborepo to the specified workspace. A non-zero exit code from ESLint fails this stage and stops the pipeline.

3.4 Typecheck

```
- run: npx turbo run typecheck --filter=<app-name>
```

Runs tsc --noEmit against the target application. Type errors fail this stage. The typecheck task is defined in each application's package.json and extends the shared tsconfig.base.json.

3.5 Unit Tests

```
- run: npx turbo run test --filter=<app-name>
```

Runs the Vitest suite for the target application. A failing test or a coverage threshold violation fails this stage. Test results are uploaded as a GitHub Actions artifact for inspection without downloading.

3.6 Build

```
- run: npx turbo run build --filter=<app-name>
```

Produces the production build artefact in the application's dist/ directory. A build error fails this stage. The dist/ directory is not committed to the repository. It is consumed by the deploy stage within the same pipeline run.

3.7 Deploy to Vercel

```
- run: npx vercel --prod --token=${{ secrets.VERCEL_TOKEN }}
  env:
    VERCEL_ORG_ID: ${{ secrets.VERCEL_ORG_ID }}
    VERCEL_PROJECT_ID: ${{ secrets.<APP>_PROJECT_ID }}
```

Deploys the build artefact to the application's Vercel project using the Vercel CLI. Each application uses its own VERCEL_PROJECT_ID secret. VERCEL_TOKEN and VERCEL_ORG_ID are shared across all pipelines.

This stage runs only on pushes to main. Pull request runs skip this stage and end after the build stage.

3.8 E2E Suite (Shell Pipeline Only)

```
- run: npx playwright test
  working-directory: playwright/
```

Triggers the full Playwright E2E suite against the production shell URL immediately after the shell deployment completes. This stage is defined only in shell.yml. It does not appear in inventory.yml, orders.yml, or analytics.yml.

A Playwright test failure causes this stage to exit with a non-zero code, which marks the shell pipeline run as failed. The deployment itself has already completed: the failure indicates that the deployed version did not pass acceptance testing. The on-call engineer is notified via Slack.

4. Required Secrets

Secret Name	Scope	Description
VERCEL_TOKEN	All pipelines	Vercel personal access token with deploy permissions
VERCEL_ORG_ID	All pipelines	Vercel organisation ID for the Modulus team account
SHELL_PROJECT_ID	shell.yml only	Vercel project ID for modulus-shell
INVENTORY_PROJECT_ID	inventory.yml only	Vercel project ID for modulus-inventory
ORDERS_PROJECT_ID	orders.yml only	Vercel project ID for modulus-orders
ANALYTICS_PROJECT_ID	analytics.yml only	Vercel project ID for modulus-analytics
SLACK_WEBHOOK_URL	shell.yml only	Incoming webhook URL for the engineering

Secret Name	Scope	Description
		Slack channel
PAGES_TOKEN	shell.yml only	GitHub token with pages:write permission for report publishing

5. Environment Variables

The shell application requires three environment variables at build time to resolve remote URLs. These are set in the Vercel project dashboard for the modulus-shell project, not in the GitHub Actions secrets.

Variable	Value	Set In
VITE_INVENTORY_URL	https://modulus-inventory.vercel.app	Vercel modulus-shell project settings
VITE_ORDERS_URL	https://modulus-orders.vercel.app	Vercel modulus-shell project settings
VITE_ANALYTICS_URL	https://modulus-analytics.vercel.app	Vercel modulus-shell project settings

6. Branch Protection Rules

The main branch is protected with the following rules configured in the GitHub repository settings:

- Require status checks to pass before merging: all four pipeline workflows must pass
- Require branches to be up to date before merging: prevents merging stale branches
- Require at least one approving review: enforces code review for all production changes
- Dismiss stale reviews when new commits are pushed: prevents approval of outdated code
- Do not allow force pushes to main
- Do not allow deletion of main

7. Full Shell Pipeline YAML

The complete shell.yml workflow for reference:

```
name: Shell CI/CD
```

```

on:
  push:
    branches: [main]
    paths: ["apps/shell/**"]
  pull_request:
    branches: [main]
    paths: ["apps/shell/**"]

jobs:
  pipeline:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: "npm"

      - run: npm ci

      - run: npx turbo run lint --filter=shell

      - run: npx turbo run typecheck --filter=shell

      - run: npx turbo run test --filter=shell

      - run: npx turbo run build --filter=shell

      - name: Deploy to Vercel
        if: github.ref == "refs/heads/main"
        run: npx vercel --prod --token=${{ secrets.VERCEL_TOKEN }}
        env:
          VERCEL_ORG_ID: ${{ secrets.VERCEL_ORG_ID }}
          VERCEL_PROJECT_ID: ${{ secrets.SHELL_PROJECT_ID }}

      - name: Run E2E Suite
        if: github.ref == "refs/heads/main"
        working-directory: playwright/
        run: npx playwright test
        env:
          PLAYWRIGHT_BASE_URL: https://modulus-shell.vercel.app

      - name: Publish E2E Report to GitHub Pages
        if: always()
        uses: peaceiris/actions-gh-pages@v3
        with:
          github_token: ${{ secrets.PAGES_TOKEN }}
          publish_dir: playwright/playwright-report
          destination_dir: e2e-reports/${{ github.run_id }}

```

```

- name: Slack Alert on Failure
  if: failure()
  uses: slackapi/slack-github-action@v1
  with:
    payload: |
      {
        "text": "E2E failure in Modulus Shell pipeline.",
        "blocks": [{
          "type": "section",
          "text": {
            "type": "mrkdwn",
            "text": "*Modulus E2E Failure*\nRun: ${ github.run_id }\nReport: https://..."
          }
        }]
      }
  env:
    SLACK_WEBHOOK_URL: ${ secrets.SLACK_WEBHOOK_URL }
    SLACK_WEBHOOK_TYPE: INCOMING_WEBHOOK

```